Python Basics - Lab #2 of 3

This is the second lab in a three-part series that covers the fundamentals of Python. In this lab, we will focus on various Python data types, functions, and how to use IF and ELSE commands to perform conditional actions.

Python Data Types

In this section, we will explore different Python data types, including integer, floating-point, string, boolean, and list. We will be using the interactive interpreter in this section.

1. Let's start by working with integers, which in Python is simply a whole number. I'll provide a list of mathematical equations below for you to input into Python to find the solutions:
    - 5*5
    - x=4
    - x*3 equals
    - 30/3
    - 12+4
    - 4**2
        - *(This is 4 raised to the power of 2.)*
    - type(4)

```
kwtrain~%python3.13
Python 3.13.0 (v3.13.0:60403a5409f, Oct  7 2024, 00:37:40) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 5*5
25
>>> x=4
>>> x*3
12
>>> 30/3
10.0
>>> 12+4
16
>>> 4**2
16
>>> type(4)
<class 'int'>
```

2. Here, we'll examine a floating-point value, which is any number that includes a decimal point. Type `type(4.5)` and press enter.

```
>>> type(4.5)
<class 'float'>
>>>
```

3. We are going to learn about string data types. A string is a sequence of characters enclosed in quotation marks. We are going to plug in a couple of commands here.
    - type("First Name")
    - "First Name"*4

Copyright 2024, Kevin Wallace Training

- print(“first name and last name”)
- firstName=”First Name”
- space=” “
- lastName=”Last Name”
- print(firstName+space+lastName)

```
>>> type("Kevin")
<class 'str'>
>>> "Kevin"*4
'KevinKevinKevinKevin'
>>> print("Kevin Wallace")
Kevin Wallace
>>> firstName="Kevin"
>>> space=" "
>>> lastName="Wallace"
>>> print(firstName+space+lastName)
Kevin Wallace
>>>
```

4. Now, let's dive into boolean data types. A boolean data type says whether something is true or false. In this section, we'll input a series of commands:
   - ‘type(True)’
     - *(The 'T' in "True" will always be uppercase, with the remaining letters in lowercase. Likewise, the 'F' in "False" will be uppercase, and the rest of the word will be lowercase. This command indicates that we're working with a boolean data type.)*
   - firstName==”First Name”
     - *(We already assigned our first name using the string data type, so this command will display as true.)*
     - *(If we used just one equal sign in this command, it would assign our first name to the `firstName` variable. However, when performing a test or evaluation, we need to use two equal signs.)*
   - firstName==”Last Name”
   - firstName!=”Last Name”
     - *(`!=` represents "not equal to".)*
   - 5<=4
     - *(‘<=’ represents “less than or equal to”.)*
   - 3<3
   - 3<=3

```
>>> type(True)
<class 'bool'>
>>> firstName=="Kevin"
True
>>> firstName=="Wallace"
False
>>> firstName!="Wallace"
True
>>> 3<3
False
>>> 3<=3
True
>>>
```

Copyright 2024, Kevin Wallace Training

5. Now, we'll discuss the list data type. Up until now, we've been assigning a single value to a single variable, but now we'll focus on storing multiple items within one variable. Let's practice with the examples below:
    - inventory=["Catalyst 3750","Catalyst 2960","2911 Router","7965G IP Phone"]
        - *(To store items in a list variable, we enclose them in open and closed brackets "[ ]".)*
        - *(We'll enclose our inventory items in quotes since they are strings, and note that these items are comma separated values as well.)*
    - *type(inventory)*

```
>>> inventory=["Catalyst 3750","Catalyst 2960","2911 Router","7965G IP Phone"]
>>> type(inventory)
<class 'list'>
>>>
```

Python Functions

In this section, we'll explore several Python functions that take an input and produce an output. The functions we'll cover include type, print, len, append, and input. We'll continue working in the interactive interpreter environment.

1. First, let's focus on the type function in Python. This function helps us determine the data type of a given value. Below are some examples:
    a. type(4)
        i. *(This is representative of an integer data type.)*
    b. type(4.5)
        i. *(This is representative of a floating-point value.)*
    c. type("First Name")
        i. *(This is representative of a string data type.)*
    d. type(True)
        i. *(This is representative of a boolean data type.)*
    e. type(inventory)
        i. *(This is representative of the list data type that we created in the last section.)*

```
>>> type(4)
<class 'int'>
>>> type(4.5)
<class 'float'>
>>> type("Kevin")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type(inventory)
<class 'list'>
>>>
```

2. Next, we are going to focus on the print function. Here is an example below:

Copyright 2024, Kevin Wallace Training

a. print("Your first and last name")
    i. *(Notice how what we put in quotes was printed on the screen.)*

```
>>> print("Kevin Wallace")
Kevin Wallace
>>>
```

3. Now, we are going to focus on the len (length) function. This will show us how many characters are in a string or how many items are in an inventory. Here are some examples below:
    a. len("First Name")
        i. *(This will display the amount of characters we have in our first name.)*
    b. len(inventory)
        i. *(This will display the number of items in the inventory we created earlier using the list data type.)*

```
>>> len("Kevin")
5
>>> len(inventory)
4
>>>
```

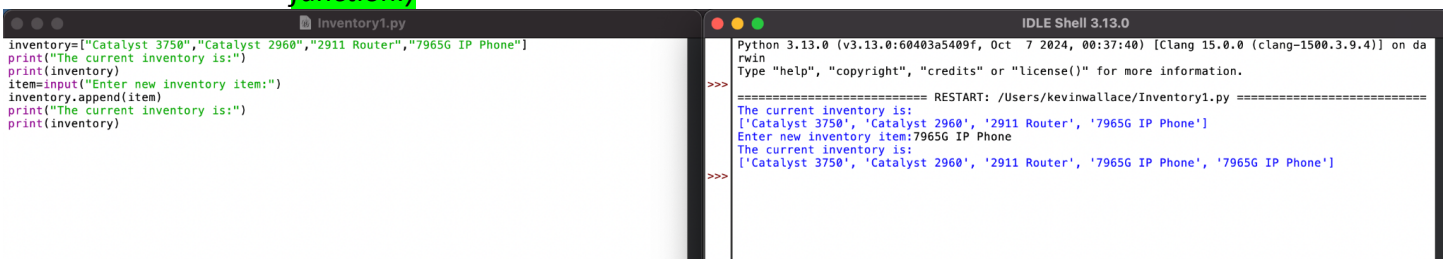
4. Let's focus on another function called append. Append allows us to add a new item to a list. See the example below:
    a. inventory.append("CUCME Server")
        i. *("Inventory" is the name of the list we created earlier.)*
    b. Inventory
        i. *(If we type "inventory" after running the above command, we will see that "CUCME Server" has been added to our list.)*

```
>>> inventory.append("CUCME Server")
>>> inventory
['Catalyst 3750', 'Catalyst 2960', '2911 Router', '7965G IP Phone', 'CUCME Server']
>>>
```

5. Finally, let's examine the input function. This function allows us to prompt the user for additional items. In this example, we will be working within the IDLE environment to recreate the list we made in the first section of this lab. Once the list is created, we will use the input function to add a new item to that list.
    a. inventory=["Catalyst 3750","Catalyst 2960","2911 Router","7965G IP Phone"]
    b. print("The current inventory is:")
    c. print(inventory)
        i. *(The print function here will print out what is currently in the inventory.)*
    d. item=input("Enter new inventory item:")
        i. *(This is the point where we are asking the user to input/add a new inventory item.)*
    e. inventory.append(item)
        i. *(This is where we take the item that the user gave us and add it to the current inventory.)*
    f. print("The current inventory is:")

Copyright 2024, Kevin Wallace Training

g. print(inventory)
   i. *(Save the file.)*
   ii. *(Click on Run, and then Run Module.)*
h. 7965G IP Phone
   i. *(This is where you will enter in the new inventory item. After you hit enter, you will see an updated list of our inventory.)*
   ii. *(In the screenshot below, you can see that the second 7965G IP Phone has been added to our inventory list, which was done using the input function.)*



Performing Conditional Actions with the IF and ELSE Commands

When working with boolean operations, we often need to check a condition and, depending on whether the condition is true or false, specify different outcomes. In other words, we might say, "IF this condition is true, this should happen; if it's false, something ELSE should happen." We are going to be using the interactive interpreter in this section.

1. First, we'll look at the IF command. What we're expressing here in this example is, "If this statement is True, then I want the prompt to display something."
   a. If 10<100:
      i. *(When we type in a colon and press enter, the prompt changes, enabling us to enter a compound instruction with multiple lines of instruction.)*
   b. print("10 is less than 100.")
      i. *(This command requires an indentation. In my example, I used 4 spaces, but you can use 3 or 5 if you prefer.)*
      ii. *(To execute this command, we have to press enter twice.)*
      iii. *(If the boolean test had come out as False instead of True, nothing would have happened in this example.)*

```
>>> if 10<100:
...     print("10 is less than 100.")
...
10 is less than 100.
>>>
```

Copyright 2024, Kevin Wallace Training

2. Now, we are going to use both the IF and ELSE commands. And what we are trying to say in this example is, "IF this condition is true, this should happen; if it's false, something ELSE should happen."
    a. nativeVLAN=1
    b. dataVLAN=100
    c. If dataVLAN==nativeVLAN:
        i. *(Remember, two equal signs represent "is equal to.")*
    d. print("The Native and Data VLANS are the same.")
    e. else:
        i. *(The indented statement above will only be executed if the condition that the Native and Data VLANs are equal is true.)*
        ii. *(Here, we are saying, "do something ELSE if this statement is not true.)*
    f. print("The Native and Data VLANS are different.")
        i. *(Now, let's hit enter twice to see what our results are.)*

```
>>> nativeVLAN=1
>>> dataVLAN=100
>>> if dataVLAN==nativeVLAN:
...     print("The Native and Data VLANS are the same.")
... else:
...     print("The Native and Data VLANS are different.")
...
The Native and Data VLANS are different.
>>>
```

We will continue our learning in part 3 of this Python series…

Copyright 2024, Kevin Wallace Training